

Notification Service using OOP and Dependency Injection

Python

```
"""

from typing import Protocol

import logging

# -----

# Setup Logging (Friendly console output for learning)

# -----

logging.basicConfig(

    level=logging.INFO,

    format="%asctime)s [%levelname)s] %(message)s"

)

logger = logging.getLogger(__name__)
```

```

# -----

# Interfaces (What senders should be able to do)

# -----

class EmailSender(Protocol):

    """Any email sender must implement this contract."""

    def send_email(self, recipient: str, subject: str, body: str) ->
bool: ...

class SMSSender(Protocol):

    """Contract for SMS providers."""

    def send_sms(self, phone_number: str, message: str) -> bool: ...

class PushNotificationSender(Protocol):

```

```

        """Contract for mobile push services."""

        def send_push(self, device_id: str, title: str, message: str) ->
bool: ...


# -----

# Concrete Implementations (Real-world providers)

# -----


class SMTPEmailSender:

    """SMTP-based email sender (e.g., Gmail, Outlook)."""

    def __init__(self, smtp_server: str, port: int, username: str,
password: str):

        self.smtp_server = smtp_server

        self.port = port

        self.username = username

```

```

        self.password = password

    def send_email(self, recipient: str, subject: str, body: str) ->
bool:

        try:

            logger.info(f"Sending email to {recipient} via SMTP...")

            # In production: use smtplib here

            print(f"""

----- EMAIL -----

To      : {recipient}

Subject : {subject}

{body}

-----

""")

            return True

        except Exception as ex:

```

```

        logger.error(f"Failed to send email: {ex}")

    return False


class TwilioSMSsender:

    """Twilio-powered SMS sender."""

    def __init__(self, account_sid: str, auth_token: str,
from_number: str):

        self.account_sid = account_sid

        self.auth_token = auth_token

        self.from_number = from_number

    def send_sms(self, phone_number: str, message: str) -> bool:

        try:

            logger.info(f"Sending SMS to {phone_number}")

            print(f"""

```

----- SMS -----

From : {self.from_number}

To : {phone_number}

{message}

""")

return True

except Exception as ex:

logger.error(f"Failed to send SMS: {ex}")

return False

class FirebasePushSender:

"""Firebase Cloud Messaging for push notifications."""

```

def __init__(self, application_name: str):

    self.application_name = application_name


def send_push(self, device_id: str, title: str, message: str) ->
bool:

    try:

        logger.info(f"Sending push to device {device_id}")

        print(f"""

----- PUSH -----

App      : {self.application_name}

Device   : {device_id}


Title    : {title}

Message  : {message}

-----

""")

        return True

```

```

except Exception as ex:

    logger.error(f"Failed to send push: {ex}")

    return False


# -----

# Core Service (High-level abstraction)

# -----


class NotificationService:

    """Central service that orchestrates notifications.

    Depends only on abstractions – easy to test and extend!"""

    def __init__(

        self,

        email_sender: EmailSender,

```



```

        sms_sender: SMSSender,

        push_sender: PushNotificationSender,

    ):

        self.email_sender = email_sender

        self.sms_sender = sms_sender

        self.push_sender = push_sender


    def send_email_notification(self, recipient: str, subject: str,
body: str) -> bool:

        """Send a single email."""

        return self.email_sender.send_email(recipient, subject,
body)


    def send_sms_notification(self, phone_number: str, message: str)
-> bool:

        """Send a single SMS."""

        return self.sms_sender.send_sms(phone_number, message)

```

```

def send_push_notification(self, device_id: str, title: str,
message: str) -> bool:

    """Send a single push notification."""

    return self.push_sender.send_push(device_id, title, message)


def send_all_notifications(

    self,

    email: str,

    phone: str,

    device: str,

    subject: str,

    email_body: str,

    sms_message: str,

    push_title: str,

    push_message: str,

) -> bool:

```

```

        """Convenience method to blast all channels."""

        email_ok = self.send_email_notification(email, subject,
email_body)

        sms_ok = self.send_sms_notification(phone, sms_message)

        push_ok = self.send_push_notification(device, push_title,
push_message)

        return email_ok and sms_ok and push_ok


# -----

# Mocks for Testing (Super useful!)

# -----


class MockEmailSender:

    def send_email(self, recipient: str, subject: str, body: str) ->
bool:

```

```
logger.info("Mock: Email sender called")
```

```
return True
```

```
class MockSMSSender:
```

```
    def send_sms(self, phone_number: str, message: str) -> bool:
```

```
        logger.info("Mock: SMS sender called")
```

```
        return True
```

```
class MockPushSender:
```

```
    def send_push(self, device_id: str, title: str, message: str) ->
```

```
bool:
```

```
        logger.info("Mock: Push sender called")
```

```
        return True
```

```
# -----  
  
# Run a Friendly Demo  
  
# -----  
  
if __name__ == "__main__":  
  
    print("Starting Notification Service Demo...\n")  
  
    # Real providers (configure with your credentials)  
  
    email_sender = SMTPEmailSender(  
  
        smtp_server="smtp.gmail.com",  
  
        port=587,  
  
        username="admin@example.com",  
  
        password="password",  
  
    )  
  
    sms_sender = TwilioSMSsender(  
  
        account_sid="ACCOUNT_SID",
```

```

        auth_token="AUTH_TOKEN",

        from_number="+254700000000",

    )

    push_sender = FirebasePushSender(application_name="Bank Mobile
App")

# Inject dependencies

notification_service = NotificationService(

    email_sender=email_sender,

    sms_sender=sms_sender,

    push_sender=push_sender,

)

# Individual notifications

notification_service.send_email_notification(

    recipient="customer@example.com",

```

```
subject="Account Credited",  
  
body="KES 15,000 has been credited to your account.",  
  
)
```

```
notification_service.send_sms_notification(  
  
    phone_number="+254712345678",  
  
    message="KES 15,000 credited successfully.",  
  
)
```

```
notification_service.send_push_notification(  
  
    device_id="DEVICE12345",  
  
    title="Transaction Alert",  
  
    message="Your account has been credited.",  
  
)
```

```
# All at once
```

```
notification_service.send_all_notifications(  
  
    email="customer@example.com",  
  
    phone="+254712345678",  
  
    device="DEVICE12345",  
  
    subject="Loan Approved",  
  
    email_body="Congratulations! Your loan has been approved.",  
  
    sms_message="Your loan has been approved.",  
  
    push_title="Loan Approval",  
  
    push_message="Open the Banking App for details.",  
  
)  
  
print("\Demo complete! Try swapping providers or using mocks in  
your tests.")
```

Reflections on the Notification Service Code

The above implementation code demonstrated the core OOP and SOLID principles such as:

1. Abstraction & Interface Segregation (ISP)
2. Dependency Inversion Principle (DIP)
3. Polymorphism
4. Single Responsibility Principle (SRP)
5. Encapsulation
6. Testability & Mocking

References

Fowler, M. (2004) *Inversion of Control Containers and the Dependency Injection pattern*.

Available at: <https://martinfowler.com/articles/injection.html> (Accessed: 7 July 2026).

Gamma, E. et al. (1995) *Design patterns: elements of reusable object-oriented software*.

Addison-Wesley.

Martin, R.C. (2000) 'Design principles and design patterns', *Object Mentor*, pp. 1–20.

Martin, R.C. (2018) *Clean architecture: a craftsman's guide to software structure and design*.

Prentice Hall.

Freeman, S. and Pryce, S. (2010) *Growing object-oriented software, guided by tests*.

Addison-Wesley.